Two-Factor Authentication (2FA) with OTP — A Developer's Guide

Understand how 2FA works and build a simple OTP-based authentication flow

Table of Contents

1. Introduction
2. What is Two-Factor Authentication?
3. Why Do We Need 2FA?
4. What Does OTP Bring to the Table?
5. Types of OTP-Based Authentication
6. What We Are Building
7. Technology Stack
8. Step-by-Step Implementation
9. Creating the 2FA Screen
10. Masking the Phone Number
11. Verify the OTP
12. Successful Authentication
13. Resend OTP
14. Complete Demo Code
15. Project Structure
16. How Production 2FA Is Different
17. SMS OTP vs Authenticator Apps
18. Common Mistakes Developers Make
19. Security Checklist
20. Key Takeaways

1. Introduction

Authentication is the process of verifying that a user is who they claim to be. Traditionally, this has been done using a combination of a username and a password. However, relying solely on username/password authentication can be highly risky. If a user's password is leaked in a

database breach, guessed easily due to poor password habits, or stolen via malware, malicious actors gain unimpeded access to their account and sensitive data.
To combat this, modern applications add another layer of verification. This brings us to **Two-Factor Authentication (2FA)**.
In simple terms, 2FA requires two independent verification steps before granting a user access to an application. For example:

- **Something you know** → Your password.
- **Something you have** → Your mobile phone or an authenticator device.

With 2FA enabled, even if an attacker manages to obtain your password, they still cannot access your account because they do not have your physical device to complete the second step.

2. What is Two-Factor Authentication?

Two-Factor Authentication (2FA) is a specific subset of Multi-Factor Authentication (MFA). While MFA can require two, three, or more factors, 2FA strictly requires exactly two. These factors must come from different categories to be effective. The most common authentication factors include:

- **Something you know:** A password, a PIN code, or answers to security questions.
- **Something you have:** A smartphone, a physical hardware security key (like a YubiKey), or an authenticator app.
- **Something you are:** Biometric verification such as fingerprint scanning, face recognition, or iris scanning.

For 2FA to be truly secure, it *must* use two different categories. Using two passwords (two things you know) is not true 2FA.
Real-world examples:

- **Google/Gmail:** After entering your password, Google prompts you to tap "Yes" on your smartphone.
- **Banking Apps:** After entering a PIN, the app sends a 6-digit code to your phone via SMS.
- **GitHub:** After your password, you must enter a time-based code from an Authenticator app.

3. Why Do We Need 2FA?

2FA helps mitigate a wide array of modern security threats by ensuring a password is no longer the *only* thing standing between an attacker and a user account. It specifically defends against:

- **Password Leaks:** Breaches at major companies expose billions of passwords.
- **Credential Stuffing:** Attackers use automated scripts to try leaked passwords from one site across hundreds of other sites.
- **Password Reuse:** Users notoriously use the same password everywhere; 2FA stops an attacker even if they have the reused password.
- **Phishing:** Deceptive emails tricking users into handing over their passwords.
- **Brute-Force Attacks:** Guessing passwords systematically.

However, it is crucial to understand that **2FA does NOT make an account completely hack-proof.** Attackers can still bypass 2FA using sophisticated techniques such as real-time phishing proxies (stealing the session cookie), session hijacking, or SIM swapping (tricking a telecom provider into assigning the victim's phone number to the attacker's SIM card to intercept SMS codes).

4. What Does OTP Bring to the Table?

OTP stands for **One-Time Password**. An OTP is an automatically generated numeric or alphanumeric string of characters that authenticates a user for a single transaction or login session.
It is called "one-time" because once the code is successfully used to log in, it becomes immediately invalid. Furthermore, OTPs are typically short-lived (expiring in 5 to 10 minutes) so that if an attacker intercepts the code but doesn't use it immediately, it becomes useless. Generating a new OTP should also instantly invalidate any previously generated, unused OTPs to prevent confusion and reduce the attack surface.

The Typical OTP Flow

```
User enters username + password
            ↓
Server verifies credentials
            ↓
Generate OTP
            ↓
Send OTP to user's second factor
            ↓
User enters OTP
            ↓
Server verifies OTP
            ↓
Authentication successful
```

5. Types of OTP-Based Authentication

There are several ways an OTP can be delivered or generated:

- **SMS OTP:** The code is sent directly to the user's mobile phone via text message. *(Advantage: Familiar and doesn't require installing apps. Disadvantage: Vulnerable to SIM-swapping and network interception.)*
- **Email OTP:** The code is sent to the user's registered email address. *(Advantage: Easy to implement and free. Disadvantage: If the email account itself is compromised, the 2FA is compromised.)*
- **TOTP (Time-based One-Time Password):** The code is generated continuously (usually every 30 seconds) by an authenticator app (like Google Authenticator, Authy). *(Advantage: Works offline, not vulnerable to SIM swapping. Disadvantage: Requires user setup and app installation.)*

For the scope of this tutorial, we are demonstrating an OTP-based 2FA flow using a mocked OTP generator in the browser, rather than sending a real SMS.

6. What We Are Building

We will build a simple React application that simulates a standard 2FA login experience. The demo flow is as follows:

```
Login Page
   ↓
Username + Password
   ↓
Login
   ↓
Generate OTP
   ↓
2FA Verification Screen
   ↓
Enter 6-digit OTP
   ↓
Verify OTP
   ↓
Dashboard
```

⚠️ **IMPORTANT — DEMO ONLY:** This is strictly an educational frontend-only demonstration. This demo does NOT: send real SMS messages, store users in a database,

use backend auth providers like Firebase/Twilio/MSG91, or implement production-ready authentication. The OTP is generated locally and logged to the browser console solely for demonstration purposes. This implementation should **not** be used for production authentication.

7. Technology Stack

- **React:** The UI library used to build our interactive user interfaces.
- **TypeScript:** Adds static typing to JavaScript, making our code more robust and catching errors early.
- **Vite:** A lightning-fast modern build tool and development server.
- **Tailwind CSS v4:** A utility-first CSS framework that allows us to style our application quickly without writing custom CSS files.
- **JavaScript Web APIs:** Used for managing timers (like `setInterval` for the OTP resend countdown) and frontend state.

8. Step-by-Step Implementation

Step 1 — Create the React Project

Run the following commands in your terminal to bootstrap a new Vite + React + TypeScript project:

```
npm create vite@latest 2fa-demo -- --template react-ts
cd 2fa-demo
npm install
npm run dev
```

These commands scaffold the project, navigate into the directory, install default dependencies, and start the local development server.

Step 2 — Set Up Tailwind CSS v4

Tailwind CSS v4 simplifies configuration. Install the required packages:

```
npm install tailwindcss @tailwindcss/vite
```

Next, update your vite.config.ts to include the Tailwind plugin:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
})
```

Finally, replace the contents of your src/index.css with:

```
@import "tailwindcss";
```

Step 3 — Create the Login Screen

The login screen requires state for the username and password, as well as a form submission handler to validate the inputs.

```
import { useState } from 'react';

export default function Login({ onLogin }: { onLogin: () => void })
{
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    if (!username || !password) {
      setError('Please fill in both fields.');
      return;
    }
    // In a real app, verify credentials via backend here.
    onLogin();
  };

  return (
    <div className="flex flex-col items-center justify-center
h-screen bg-gray-50">
      <form onSubmit={handleSubmit} className="p-8 bg-white
shadow-lg rounded-lg w-96">
        <h2 className="text-2xl font-bold mb-6
text-center">Login</h2>
        {error && <p className="text-red-500 mb-4
```

```
text-sm">{error}</p>}
        <input
          type="text" placeholder="Username"
          value={username} onChange={(e) =>
setUsername(e.target.value)}
          className="w-full mb-4 p-2 border border-gray-300
rounded" />
        <input
          type="password" placeholder="Password"
          value={password} onChange={(e) =>
setPassword(e.target.value)}
          className="w-full mb-6 p-2 border border-gray-300
rounded" />
        <button type="submit" className="w-full bg-blue-600
text-white p-2 rounded hover:bg-blue-700">
          Login
        </button>
      </form>
    </div>
  );
}
```

Step 4 — Generate a Demo OTP

OTP generation requires creating a random 6-digit number. We convert it to a string because OTPs often start with a '0' (though our math ensures a range of 100000-999999 for simplicity) and because string comparison is safer for inputs.

```
const generateOTP = () => {
  const otp = Math.floor(100000 + Math.random() *
900000).toString();
  console.log("Demo OTP:", otp);
  return otp;
};
```

🔐 **SECURITY NOTE:** Logging the OTP to the console is only being done because this is a tutorial/demo. A production application must **never** expose OTPs in the browser console, network responses, or client-side code.

9. Creating the 2FA Screen

We need a dedicated screen with 6 individual input boxes that auto-advances the cursor as the user types.

```
// In a real scenario, this logic maps over an array of 6 elements.
// Users can type a number, and the focus moves to the next input automatically.
// Backspace removes a number and moves focus to the previous input.
```

(See Section 14 for the complete React code for this component)

10. Masking the Phone Number

Applications should avoid displaying complete phone numbers or email addresses on the verification screen to protect user privacy. Only the final few digits are displayed so the user can verify the destination without exposing the entire number.

```
const maskPhoneNumber = (phone: string) => {
  return `•••••••••• ${phone.slice(-4)}`;
};

// Usage: maskPhoneNumber("+19876544821") → "•••••••••• 4821"
```

11. Verify the OTP

When the user clicks "Verify", we simply compare the entered OTP string with the generated OTP string.

```
if (enteredOTP === generatedOTP) {
  // Authentication successful
  onSuccess();
} else {
  // Invalid OTP
  setError('Invalid OTP. Please try again.');
}
```

In a real application, the frontend sends the enteredOTP to a backend API endpoint, which verifies the OTP against a database or cache (like Redis), ensuring it hasn't expired.

12. Successful Authentication

Only after both the password and the OTP are verified should the user be directed to the dashboard. True authentication only concludes when all required factors are passed.

```
export default function Dashboard() {
  return (
    <div className="text-center mt-20">
      <h1 className="text-3xl font-bold text-green-600
mb-4">Welcome back 👋</h1>
      <p className="text-lg">✓ Username & Password Verified</p>
      <p className="text-lg">✓ Two-Factor Authentication
Verified</p>
    </div>
  );
}
```

13. Resend OTP

Networks can delay SMS delivery, so applications need a "Resend OTP" option. To prevent abuse, this feature must include a countdown timer.

- Generate a new OTP and replace the previous one.
- Clear the input fields.
- Start a 30-second countdown before the user can request another one.

In production, rate limiting (e.g., max 3 resends per hour) is critical to prevent attackers from racking up huge SMS gateway bills (a common exploit known as toll fraud).

14. Complete Demo Code

Here is a combined React component demonstrating the full flow. In a real project, these would be split into separate files.

```
import { useState, useEffect, useRef } from 'react';

type Step = 'LOGIN' | 'OTP' | 'DASHBOARD';

export default function App() {
  const [step, setStep] = useState<Step>('LOGIN');
```

```
  const [generatedOTP, setGeneratedOTP] = useState('');

  // OTP generation logic
  const triggerOTP = () => {
    const otp = Math.floor(100000 + Math.random() *
900000).toString();
    console.log("Demo OTP:", otp); // DEMO ONLY
    setGeneratedOTP(otp);
    setStep('OTP');
  };

  if (step === 'LOGIN') {
    return <LoginScreen onLogin={triggerOTP} />;
  }
  if (step === 'OTP') {
    return <OTPScreen expectedOTP={generatedOTP} onSuccess={() =>
setStep('DASHBOARD')} onResend={triggerOTP} />;
  }
  return <Dashboard onLogout={() => setStep('LOGIN')} />;
}

// --- LOGIN SCREEN ---
function LoginScreen({ onLogin }: { onLogin: () => void }) {
  const [user, setUser] = useState('');
  const [pass, setPass] = useState('');
  return (
    <div className="flex flex-col items-center mt-20">
      <h2 className="text-2xl font-bold mb-4">Login</h2>
      <input className="border p-2 mb-2 rounded"
placeholder="Username" value={user} onChange={e =>
setUser(e.target.value)} />
      <input className="border p-2 mb-4 rounded" type="password"
placeholder="Password" value={pass} onChange={e =>
setPass(e.target.value)} />
      <button onClick={onLogin} className="bg-blue-600 text-white
px-4 py-2 rounded">Login</button>
    </div>
  );
}

// --- OTP SCREEN ---
function OTPScreen({ expectedOTP, onSuccess, onResend }: {
expectedOTP: string, onSuccess: () => void, onResend: () => void })
{
  const [otp, setOtp] = useState(new Array(6).fill(""));
  const [timer, setTimer] = useState(30);
  const [error, setError] = useState('');
  const inputRefs = useRef<(HTMLInputElement | null)[]>([]);
```

```
  useEffect(() => {
    timer > 0 && setTimeout(() => setTimer(timer - 1), 1000);
  }, [timer]);

  const handleChange = (e: React.ChangeEvent<HTMLInputElement>,
index: number) => {
    const val = e.target.value;
    if (isNaN(Number(val))) return;
    const newOtp = [...otp];
    newOtp[index] = val.substring(val.length - 1);
    setOtp(newOtp);

    // Auto-focus next
    if (val && index < 5 && inputRefs.current[index + 1]) {
      inputRefs.current[index + 1]?.focus();
    }
  };

  const handleVerify = () => {
    if (otp.join("") === expectedOTP) {
      onSuccess();
    } else {
      setError('Invalid OTP!');
    }
  };

  const handleResend = () => {
    setOtp(new Array(6).fill(""));
    setTimer(30);
    setError('');
    onResend();
  };

  return (
    <div className="flex flex-col items-center mt-20">
      <h2 className="text-2xl font-bold mb-2">Enter Verification
Code</h2>
      <p className="text-gray-600 mb-6">Sent to •••••••••• 4821</p>
      <div className="flex gap-2 mb-6">
        {otp.map((data, index) => (
          <input
            key={index}
            type="text"
            className="w-12 h-12 text-center border rounded
text-xl"
            value={data}
            onChange={e => handleChange(e, index)}
```

```
            ref={el => inputRefs.current[index] = el}
          />
        ))}
      </div>
      {error && <p className="text-red-500 mb-4">{error}</p>}
      <button onClick={handleVerify} className="bg-green-600
text-white px-8 py-2 rounded mb-4">Verify</button>
      <button disabled={timer > 0} onClick={handleResend}
className={`text-sm ${timer > 0 ? 'text-gray-400' : 'text-blue-600
underline'}`}>
        {timer > 0 ? `Resend OTP in ${timer}s` : 'Resend OTP now'}
      </button>
    </div>
  );
}

// --- DASHBOARD ---
function Dashboard({ onLogout }: { onLogout: () => void }) {
  return (
    <div className="flex flex-col items-center mt-20">
      <h1 className="text-3xl font-bold text-green-600
mb-4">Welcome back 👋</h1>
      <p className="mb-2">✓ Username & Password Verified</p>
      <p className="mb-6">✓ Two-Factor Authentication Verified</p>
      <button onClick={onLogout} className="bg-red-500 text-white
px-4 py-2 rounded">Logout</button>
    </div>
  );
}
```

15. Project Structure

For better maintainability, organize your files into specific components:

```
src/
├── components/
│   ├── Login.tsx
│   ├── OTPVerification.tsx
│   └── Dashboard.tsx
├── App.tsx
├── main.tsx
└── index.css
```

- **components/**: Houses isolated UI pieces for better separation of concerns.
- **App.tsx**: Manages the state machine (LOGIN → OTP → DASHBOARD).
- **index.css**: Where Tailwind v4 is imported.

16. How Production 2FA Is Different

The tutorial implementation is intentionally simplified to teach frontend mechanics. A production 2FA architecture behaves very differently:

```
React Frontend
      ↓
Backend API
      ↓
Credential Verification
      ↓
Generate OTP
      ↓
OTP Storage (Redis / DB Cache)
      ↓
SMS / Email Provider (Twilio, AWS SNS, MSG91)
      ↓
User
      ↓
OTP Verification API
      ↓
Create Authenticated Session (JWT / Cookies)
```

In production:

- OTP generation **must** happen on the backend.
- The generated OTP should **never** be exposed to the frontend; the frontend simply acts as a dumb terminal sending the user's input to the backend.
- OTPs must have strict expiration times (e.g., 5-10 minutes) stored in the backend cache.
- OTPs should ideally be hashed when stored on the backend where appropriate.
- OTP attempts must be severely limited (e.g., max 3-5 failed guesses) to prevent brute-forcing a 6-digit code.
- OTP requests (resends) must be rate-limited.
- Sessions and tokens should be securely managed (e.g., HttpOnly cookies) only *after* the OTP is successfully validated.
- HTTPS must always be used to prevent interception in transit.
- Sensitive authentication events (failures, locks) must be logged securely on the backend for audit trails.

- Recovery/backup codes should be provided to users in case they lose their phone.

17. SMS OTP vs Authenticator Apps

While SMS OTP is common, security professionals prefer authenticator-based TOTP for stronger security where practical.

Feature	SMS OTP	Authenticator App (TOTP)
Requires phone network	Yes	No
Requires internet	Usually for SMS delivery	No (for generating the TOTP)
SIM-swap risk	Higher	Lower (tied to the physical device)
Setup difficulty	Easy (Requires just a phone number)	Moderate (Requires scanning a QR code)
Common use	Very common (Banking, Retail)	Common (Developer tools, Crypto, Enterprise)

18. Common Mistakes Developers Make

- **Generating OTPs on the frontend in production:** If the code is generated in the browser, an attacker can simply read it from the network/console and bypass 2FA entirely.
- **Storing OTPs permanently:** OTPs should live in temporary memory caches (like Redis) with a TTL (Time-To-Live), not permanently in user SQL databases.
- **Not expiring OTPs:** An OTP that lasts forever defeats the point of it being a "one-time" transient password.
- **Allowing unlimited OTP attempts:** Without rate-limiting, an attacker can write a script to guess all 1,000,000 combinations of a 6-digit OTP in seconds.
- **Allowing unlimited resend requests:** Leads to massive SMS API bills and user harassment.
- **Showing the full phone number:** Exposes user PII (Personally Identifiable Information)

on the screen.

- **Logging OTPs in production:** Accidentally exposing plain-text OTPs in server application logs.
- **Using predictable OTPs:** Relying on weak pseudo-random number generators instead of cryptographically secure generation on the backend.
- **Not invalidating old OTPs:** Generating a new code should immediately kill the old code.
- **Not using HTTPS:** Credentials and OTPs sent over HTTP can be easily sniffed over public Wi-Fi.
- **Treating frontend validation as security:** Client-side checks are for UX only. All security logic MUST reside on the backend.

19. Security Checklist

Before deploying 2FA to production, review this checklist:

☐ OTP generated server-side
☐ OTP expires quickly (e.g., 5 mins)
☐ OTP is single-use
☐ Old OTP invalidated after a resend
☐ Maximum verification attempts enforced
☐ OTP requests rate-limited
☐ HTTPS enabled
☐ OTP not logged in production
☐ Phone/email information masked
☐ Authentication handled server-side
☐ Secure session/token handling applied only after OTP verify
☐ Backup/recovery mechanism available

20. Key Takeaways

- **Passwords alone aren't always enough.** They are easily stolen or reused.
- **2FA adds another authentication factor.** It requires both something you know and something you have.
- **OTPs can provide a second verification step.** Delivered via SMS, email, or an authenticator app.
- **The OTP should be short-lived and single-use.** This limits the window of opportunity for attackers.
- **Production OTP generation and verification should happen on the backend.** Never

trust the client side for security operations.

- **Rate limiting and secure session management are essential.** Prevent brute force and toll-fraud abuse.
- **The frontend-only implementation in this tutorial is strictly a learning/demo implementation.**